

Program 11

Naïve Bayes Classifier (from Scratch)

Step 1: Prior Probability of Each Class

The first step in Naïve Bayes is to calculate the probability of each class from the training data.

$$P(C_k) = \frac{\text{Number of samples in class } C_k}{\text{Total number of training samples}}$$

where:

- $P(C_k)$ = prior probability of class C_k
- C_k = a class label such as 'Yes' or 'No'

For example, if the training data contains 6 samples of **Yes** and 4 samples of **No**, then:

$$P(\text{Yes}) = \frac{6}{10} = 0.6$$

$$P(\text{No}) = \frac{4}{10} = 0.4$$

Step 2: Conditional Probability of Feature Values Given a Class

For each feature value, we calculate its probability under a particular class.

$$P(X_j = x_j | C_k) = \frac{\text{Count of samples with } X_j = x_j \text{ in class } C_k}{\text{Total number of samples in class } C_k}$$

where:

- X_j = the j-th feature
- x_j = a specific value of that feature
- C_k = class label

For example:

$$P(\text{Outlook} = \text{Sunny} | \text{No})$$

means the probability that **Outlook = Sunny**, given that the class is **No**.

Step 3: Laplace Smoothing

Sometimes a feature value may not appear in a particular class, which makes its probability zero.

To avoid this problem, we use Laplace smoothing.

$$P(X_j = x_j | C_k) = \frac{N_{jk} + 1}{N_k + V_j}$$

where:

N_{jk} = count of samples where $X_j = x_j$ in class C_k N_k = total number of samples in class C_k

V_j = number of possible values of feature X_j

This ensures that no conditional probability becomes zero.

Step 4: Naïve Bayes Prediction Formula

Suppose a test sample is:

$$X = (x_1, x_2, x_3, \dots, x_n)$$

Then for each class (C_k), we compute:

$$P(C_k | X) \propto P(C_k) \prod_{j=1}^n P(x_j | C_k)$$

This means:

$$P(C_k | X) \propto P(C_k) \cdot P(x_1 | C_k) \cdot P(x_2 | C_k) \cdot P(x_3 | C_k) \dots P(x_n | C_k)$$

Here:

$P(C_k)$ = prior probability of class $P(x_j | C_k)$ = conditional probability of each feature value given the class

The word **naïve** means we assume that all features are conditionally independent given the class.

Step 5: Prediction Rule

After computing the probability score for each class, the predicted class is the one with the maximum score.

$$\hat{C} = \arg \max_{C_k} P(C_k) \prod_{j=1}^n P(x_j | C_k)$$

where:

$\hat{C}$ = predicted class label

So, the classifier chooses the class with the highest posterior probability.

Step 6: Prediction for One Test Sample

Suppose the test sample is:

- Outlook = Sunny
- Temperature = Mild
- Humidity = Normal
- Wind = Strong

Then for class **Yes**, the score is:

$$\text{Score(Yes)} = P(\text{Yes}) \cdot P(\text{Sunny} | \text{Yes}) \cdot P(\text{Mild} | \text{Yes}) \cdot P(\text{Normal} | \text{Yes}) \cdot P(\text{Strong} | \text{Yes})$$

Similarly, for class **No**, the score is:

$$\text{Score(No)} = P(\text{No}) \cdot P(\text{Sunny} | \text{No}) \cdot P(\text{Mild} | \text{No}) \cdot P(\text{Normal} | \text{No}) \cdot P(\text{Strong} | \text{No})$$

Then compare both values:

if $\text{Score(Yes)} > \text{Score(No)}$, predict Yes otherwise, predict No

Step 7: Accuracy of the Classifier

After predicting all test samples, the accuracy is computed as:

$$\text{Accuracy} = \frac{\text{Number of correctly predicted test samples}}{\text{Total number of test samples}} \times 100$$

For example, if 3 out of 4 test samples are correctly classified, then:

$$\text{Accuracy} = \frac{3}{4} \times 100 = 75\%$$

Final Formula of Naïve Bayes

The core formula used in the program is:

$$\hat{C} = \arg \max_{C_k} P(C_k) \prod_{j=1}^n P(x_j | C_k)$$

This means:

1. Calculate the prior probability of each class
 2. Calculate the conditional probability of each feature value given the class
 3. Multiply these probabilities
 4. Choose the class with the highest result
-

```

import pandas as pd

# -----
# Step 1: Create Dataset Directly in Code
# -----
data_dict = {
    "Outlook": ["Sunny", "Sunny", "Overcast", "Rain", "Rain", "Rain", "Overcast",
               "Sunny", "Sunny", "Rain", "Sunny", "Overcast", "Overcast", "Rain"],
    "Temperature": ["Hot", "Hot", "Hot", "Mild", "Cool", "Cool", "Cool",
                   "Mild", "Cool", "Mild", "Mild", "Mild", "Hot", "Mild"],
    "Humidity": ["High", "High", "High", "High", "Normal", "Normal", "Normal",
                "High", "Normal", "Normal", "Normal", "High", "Normal", "High"],
    "Wind": ["Weak", "Strong", "Weak", "Weak", "Weak", "Strong", "Strong",
            "Weak", "Weak", "Weak", "Strong", "Strong", "Weak", "Strong"],
    "Play": ["No", "No", "Yes", "Yes", "Yes", "No", "Yes",
            "No", "Yes", "Yes", "Yes", "Yes", "Yes", "No"]
}

data = pd.DataFrame(data_dict)

print("Dataset:\n")
print(data)
print()

# -----
# Step 2: Define Feature Columns and Target Column
# -----
feature_cols = list(data.columns[:-1])
target_col = data.columns[-1]

print("Feature Columns:", feature_cols)
print("Target Column :", target_col)
print()

# -----
# Step 3: Split Dataset into Training and Test Data
# First 10 rows for training
# Last 4 rows for testing
# -----
train_data = data.iloc[:10]
test_data = data.iloc[10:]

print("Training Data:\n")
print(train_data)
print()

print("Test Data:\n")
print(test_data)
print()

# -----
# Step 4: Calculate Prior Probabilities P(Class)
# -----
def calculate_priors(train_df):
    priors = {}
    total_rows = len(train_df)

    class_counts = train_df[target_col].value_counts().to_dict()

    for cls, count in class_counts.items():
        priors[cls] = count / total_rows

    return priors

```

```

# -----
# Step 5: Get All Possible Values of Each Feature
# -----
def get_feature_values(train_df):
    feature_values = {}

    for col in feature_cols:
        feature_values[col] = train_df[col].unique().tolist()

    return feature_values

# -----
# Step 6: Calculate Conditional Probabilities
# P(Feature = value | Class)
# Laplace Smoothing is used
# -----
def calculate_conditionals(train_df, feature_values):
    conditionals = {}

    class_labels = train_df[target_col].unique()

    for cls in class_labels:
        class_data = train_df[train_df[target_col] == cls]
        conditionals[cls] = {}

        for col in feature_cols:
            value_counts = class_data[col].value_counts().to_dict()
            total_class_rows = len(class_data)
            num_values = len(feature_values[col])

            for value in feature_values[col]:
                count = value_counts.get(value, 0)
                prob = (count + 1) / (total_class_rows + num_values) # Laplace Smoothing
                conditionals[cls][(col, value)] = prob

    return conditionals

# -----
# Step 7: Train Naive Bayes Model
# -----
def train_naive_bayes(train_df):
    priors = calculate_priors(train_df)
    feature_values = get_feature_values(train_df)
    conditionals = calculate_conditionals(train_df, feature_values)

    model = {
        "priors": priors,
        "feature_values": feature_values,
        "conditionals": conditionals
    }

    return model

# -----
# Step 8: Predict Class for One Row
# P(Class|X) proportional to P(Class) * P(x1|Class) * P(x2|Class) * ...
# -----
def predict(model, row):
    priors = model["priors"]
    conditionals = model["conditionals"]

    class_probabilities = {}

    for cls in priors:
        prob = priors[cls]

        for col in feature_cols:
            value = row[col]
            prob *= conditionals[cls][(col, value)]

        class_probabilities[cls] = prob

    predicted_class = max(class_probabilities, key=class_probabilities.get)
    return predicted_class

```

```

# -----
# Step 9: Test the Model on Test Data
# -----
def test_naive_bayes(model, test_df):
    actual_values = []
    predicted_values = []

    for i in range(len(test_df)):
        row = test_df.iloc[i]

        actual = row[target_col]
        predicted = predict(model, row)

        actual_values.append(actual)
        predicted_values.append(predicted)

        print(f"Test Case {i+1}")
        print("Input Data :", row[feature_cols].to_dict())
        print("Actual      :", actual)
        print("Predicted    :", predicted)
        print()

    return actual_values, predicted_values

# -----
# Step 10: Calculate Accuracy
# -----
def calculate_accuracy(actual, predicted):
    correct = 0

    for a, p in zip(actual, predicted):
        if a == p:
            correct += 1

    accuracy = (correct / len(actual)) * 100
    return accuracy

# -----
# Step 11: Train the Model
# -----
model = train_naive_bayes(train_data)

print("Prior Probabilities:\n")
print(model["priors"])
print()

print("Conditional Probabilities:\n")
for cls in model["conditionals"]:
    print("Class =", cls)
    for key, value in model["conditionals"][cls].items():
        print(f"{key} : {value:.4f}")
    print()

# -----
# Step 12: Predict Test Data
# -----
actual_values, predicted_values = test_naive_bayes(model, test_data)

# -----
# Step 13: Compute Accuracy
# -----
accuracy = calculate_accuracy(actual_values, predicted_values)

print("Actual Classes      :", actual_values)
print("Predicted Classes :", predicted_values)
print()
print("Accuracy =", accuracy, "%")

```

11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Prior Probabilities:

{'Yes': 0.6, 'No': 0.4}

Conditional Probabilities:

Class = No

('Outlook', 'Sunny') : 0.5714
('Outlook', 'Overcast') : 0.1429
('Outlook', 'Rain') : 0.2857
('Temperature', 'Hot') : 0.4286
('Temperature', 'Mild') : 0.2857
('Temperature', 'Cool') : 0.2857
('Humidity', 'High') : 0.6667
('Humidity', 'Normal') : 0.3333
('Wind', 'Weak') : 0.5000
('Wind', 'Strong') : 0.5000

Class = Yes

('Outlook', 'Sunny') : 0.2222
('Outlook', 'Overcast') : 0.3333
('Outlook', 'Rain') : 0.4444
('Temperature', 'Hot') : 0.2222
('Temperature', 'Mild') : 0.3333
('Temperature', 'Cool') : 0.4444
('Humidity', 'High') : 0.3750
('Humidity', 'Normal') : 0.6250
('Wind', 'Weak') : 0.7500
('Wind', 'Strong') : 0.2500

Test Case 1

Input Data : {'Outlook': 'Sunny', 'Temperature': 'Mild', 'Humidity': 'Normal', 'Wind': 'Strong'}
Actual : Yes
Predicted : No

Test Case 2

Input Data : {'Outlook': 'Overcast', 'Temperature': 'Mild', 'Humidity': 'High', 'Wind': 'Strong'}
Actual : Yes
Predicted : Yes

Test Case 3

Input Data : {'Outlook': 'Overcast', 'Temperature': 'Hot', 'Humidity': 'Normal', 'Wind': 'Weak'}
Actual : Yes
Predicted : Yes

Test Case 4

Input Data : {'Outlook': 'Rain', 'Temperature': 'Mild', 'Humidity': 'High', 'Wind': 'Strong'}
Actual : No
Predicted : No

Actual Classes : ['Yes', 'Yes', 'Yes', 'No']

Predicted Classes : ['No', 'Yes', 'Yes', 'No']

Accuracy = 75.0 %

```
print("Training Data:\n")
print(train_data)
print()

print("Test Data:\n")
print(test_data)
print()

# -----
# Step 4: Separate Input Features and Target
# -----
X_train = train_data[feature_cols]
y_train = train_data[target_col]

X_test = test_data[feature_cols]
y_test = test_data[target_col]

# -----
# Step 5: Convert Categorical Text Data into Numbers
# -----
from sklearn.preprocessing import OrdinalEncoder

encoder = OrdinalEncoder()

X_train_encoded = encoder.fit_transform(X_train)
X_test_encoded = encoder.transform(X_test)

print("Encoded Training Features:\n")
print(X_train_encoded)
print()

print("Encoded Test Features:\n")
print(X_test_encoded)
print()

# -----
# Step 6: Train Naive Bayes Classifier using Sklearn
# -----
from sklearn.naive_bayes import CategoricalNB

model = CategoricalNB()
model.fit(X_train_encoded, y_train)

# -----
# Step 7: Predict on Test Data
# -----
y_pred = model.predict(X_test_encoded)

print("Actual Classes :", list(y_test))
print("Predicted Classes :", list(y_pred))
print()

# -----
# Step 8: Show Test Case by Test Case Prediction
# -----
for i in range(len(X_test)):
    print(f"Test Case {i+1}")
    print("Input Data :", X_test.iloc[i].to_dict())
    print("Actual      :", y_test.iloc[i])
    print("Predicted   :", y_pred[i])
    print()

# -----
# Step 9: Compute Accuracy
# -----
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred) * 100
print("Accuracy =", accuracy, "%")
```

Training Data:

	Outlook	Temperature	Humidity	Wind	Play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes

Test Data:

	Outlook	Temperature	Humidity	Wind	Play
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Encoded Training Features:

```
[[2. 1. 0. 1.]
 [2. 1. 0. 0.]
 [0. 1. 0. 1.]
 [1. 2. 0. 1.]
 [1. 0. 1. 1.]
 [1. 0. 1. 0.]
 [0. 0. 1. 0.]
 [2. 2. 0. 1.]
 [2. 0. 1. 1.]
 [1. 2. 1. 1.]]
```

Encoded Test Features:

```
[[2. 2. 1. 0.]
 [0. 2. 0. 0.]
 [0. 1. 1. 1.]
 [1. 2. 0. 0.]]
```

Actual Classes : ['Yes', 'Yes', 'Yes', 'No']

Predicted Classes : [np.str_('No'), np.str_('Yes'), np.str_('Yes'), np.str_('No')]

Test Case 1

Input Data : {'Outlook': 'Sunny', 'Temperature': 'Mild', 'Humidity': 'Normal', 'Wind': 'Strong'}

Actual : Yes

Predicted : No

Test Case 2

Input Data : {'Outlook': 'Overcast', 'Temperature': 'Mild', 'Humidity': 'High', 'Wind': 'Strong'}

Actual : Yes

Predicted : Yes

Test Case 3

Input Data : {'Outlook': 'Overcast', 'Temperature': 'Hot', 'Humidity': 'Normal', 'Wind': 'Weak'}

Actual : Yes

Predicted : Yes

Test Case 4

Input Data : {'Outlook': 'Rain', 'Temperature': 'Mild', 'Humidity': 'High', 'Wind': 'Strong'}

Actual : No

Predicted : No

Accuracy = 75.0 %